Ved Teredesai
5023166

Experiment No. 09 .

Aim : To Build, change, and destroy AWS / GCP /Microsoft Azure/ DigitalOcean infrastructure Using Terraform.

Objective: To upskill learners in creating Infrastructure as Code (IaC) using tools like Terraform or AWS CloudFormation

Outcome: CO4

Build and Deploy Infrastructure as Code using Terraform, applying problem analysis and engineering tools to create effective solutions thereby cultivate lifelong learning to stay current with evolving infrastructure practices.

❖ **Theory :**

 What Is Infrastructure as Code?

Infrastructure as code is a means by which engineers define the computer systems and managing the operations environment in the same way one does applications or other code for general release.

Infrastructure as Code (IaC) grew as a response to the difficulty posed by utility computing and second-generation web frameworks. When you use a high-level descriptive coding language to automate the IT infrastructure, it eliminates the need for engineers to manually provision and manage servers, operating systems, database connections, storage, and other infrastructure elements when they want to develop, test, or deploy a software application.

To deploy infrastructure with Terraform:

- **Scope** - Identify the infrastructure for your project.
- **Author** - Write the configuration for your infrastructure.
- **Initialize** - Install the plugins Terraform needs to manage the infrastructure. •
Plan - Preview the changes Terraform will make to match your configuration. •
Apply - Make the planned changes.

 Plan :

The terraform plan command evaluates a Terraform configuration to determine the desired state of all the resources it declares, then compares that desired state to the real infrastructure objects being managed with the current working directory and workspace. It uses state data to determine which real objects correspond to which declared resources, and checks the current state of each resource using the relevant infrastructure provider's API.

Once it has determined the difference between the current state and the desired state, terraform plan presents a description of the changes necessary to achieve the desired state. It does not perform any actual changes to real world infrastructure objects; it only presents a plan for making changes.
Plans are usually run to validate configuration changes and confirm that the resulting actions are as expected. However, terraform plan can also save its plan as a runnable artifact, which terraform apply can use to carry out those exact changes.

 Apply :

The terraform apply command performs a plan just like terraform plan does, but then actually

carries out the planned changes to each resource using the relevant infrastructure provider's API. It asks for confirmation from the user before making any changes, unless it was explicitly told to skip approval.

By default, terraform apply performs a fresh plan right before applying changes, and displays the plan to the user when asking for confirmation. However, it can also accept a plan file produced by terraform plan in lieu of running a new plan. You can use this to reliably perform an exact set of pre-approved changes, even if the configuration or the state of the real infrastructure has changed in the minutes since the original plan was created.

 Change infrastructure :

Infrastructure is continuously evolving, and Terraform helps you manage that change. As you change Terraform configurations, Terraform builds an execution plan that only modifies what is necessary to reach your desired state.

 Destroy infrastructure :

The terraform destroy command destroys all of the resources being managed by the current working directory and workspace, using state data to determine which real world objects correspond to managed resources. Like terraform apply, it asks for confirmation before proceeding.

A destroy behaves exactly like deleting every resource from the configuration and then running an apply, except that it doesn't require editing the configuration. This is more convenient if you intend to provision similar resources at a later date.

 Steps to build an infrastructure :

1. Configure the AWS CLI from your terminal. Follow the prompts to input your AWS Access Key ID and Secret Access Key.

$ aws configure

```
creating: aws/dist/awscli/botocore/data/iot-data/
creating: aws/dist/awscli/botocore/data/iot-jobs-data/
creating: aws/dist/awscli/botocore/data/iot-managed-integrations/
creating: aws/dist/awscli/botocore/data/iotdeviceadvisor/
creating: aws/dist/awscli/botocore/data/iotevents/
creating: aws/dist/awscli/botocore/data/iotevents-data/
creating: aws/dist/awscli/botocore/data/iotfleetwise/
creating: aws/dist/awscli/botocore/data/iotsecuretunneling/
creating: aws/dist/awscli/botocore/data/iotsitewise/
creating: aws/dist/awscli/botocore/data/iotthingsgraph/
creating: aws/dist/awscli/botocore/data/iottwinmaker/
creating: aws/dist/awscli/botocore/data/iotwireless/
creating: aws/dist/awscli/botocore/data/ivs/
creating: aws/dist/awscli/botocore/data/ivs-realtime/
creating: aws/dist/awscli/botocore/data/ivschat/
creating: aws/dist/awscli/botocore/data/kafka/
creating: aws/dist/awscli/botocore/data/kafkaconnect/
creating: aws/dist/awscli/botocore/data/kendra/
creating: aws/dist/awscli/botocore/data/kendra-ranking/
creating: aws/dist/awscli/botocore/data/keyspaces/
creating: aws/dist/awscli/botocore/data/keyspacesstreams/
creating: aws/dist/awscli/botocore/data/kinesis/
creating: aws/dist/awscli/botocore/data/kinesis-video-archived-media/
creating: aws/dist/awscli/botocore/data/kinesis-video-media/
creating: aws/dist/awscli/botocore/data/kinesis-video-signaling/
creating: aws/dist/awscli/botocore/data/kinesis-video-webrtc-storage/
creating: aws/dist/awscli/botocore/data/kinesisanalytics/
creating: aws/dist/awscli/botocore/data/kinesisanalyticsv2/
creating: aws/dist/awscli/botocore/data/kinesisvideo/
creating: aws/dist/awscli/botocore/data/kms/
creating: aws/dist/awscli/botocore/data/lakeformation/
creating: aws/dist/awscli/botocore/data/lambda/
creating: aws/dist/awscli/botocore/data/launch-wizard/
```

2. Write configuration

The set of files used to describe infrastructure in Terraform is known as a Terraform configuration. You will write your first configuration to define a single AWS EC2 instance. Each Terraform configuration must be in its own working directory. Create a directory for your configuration.

$ mkdir learn-terraform-aws-instance

3. Change into the directory.
$ cd learn-terraform-aws-instance

4. Create a file to define your infrastructure.
$ touch main.tf

Open main.tf in your text editor, paste in the configuration below,provider "aws" {
  region = "ap-south-1"
}

resource "aws_key_pair" "deployer" {
  key_name   = "terraform-key"
  public_key = file("~/.ssh/id_rsa.pub")
}

resource "aws_security_group" "allow_ssh" {

```
  name = "allow_ssh"

  ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_instance" "app_server" {
  ami           = "ami-03f4878755434977f"
  instance_type = "t3.micro"

  key_name = aws_key_pair.deployer.key_name
  vpc_security_group_ids = [aws_security_group.allow_ssh.id]

  tags = {
    Name = "Terraform-Server"
  }
}
```

.

5. Initialize the directory

When you create a new configuration — or check out an existing configuration from version control — you need to initialize the directory with terraform init. Initializing a configuration directory downloads and installs the providers defined in the configuration, which in this case is the aws provider.

$ terraform init

```
ved@DESKTOP-BHPHDFO:~$ cd learn-terraform-aws-instance
ved@DESKTOP-BHPHDFO:~/learn-terraform-aws-instance$ touch main.tf
ved@DESKTOP-BHPHDFO:~/learn-terraform-aws-instance$ nano main.tf

ved@DESKTOP-BHPHDFO:~/learn-terraform-aws-instance$ nano main.tf
ved@DESKTOP-BHPHDFO:~/learn-terraform-aws-instance$ terraform init
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 3.27"...
- Installing hashicorp/aws v3.76.1...
- Installed hashicorp/aws v3.76.1 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

6. Format and validate the configuration

Format your configuration. Terraform will print out the names of the files it modified, if any. In this case, your configuration file was already formatted correctly, so Terraform won't return any file names.

$ terraform fmt

7. Validate your configuration. The example configuration provided above is valid, so Terraform will return a success message.

$ terraform validate

```
ed@DESKTOP-BHPHDFO:~/learn-terraform-aws-instance$ terraform fmt
ed@DESKTOP-BHPHDFO:~/learn-terraform-aws-instance$ terraform validate
uccess! The configuration is valid.
```

8. Create infrastructure

Apply the configuration now with the terraform apply command. Terraform will print output similar to what is shown below. We have truncated some of the output to save space.

$ terraform apply

```
Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_instance.app_server: Destroying... [id=i-01fedbb1e683827db]
aws_instance.app_server: Still destroying... [id=i-01fedbb1e683827db, 00m10s elapsed]
aws_instance.app_server: Still destroying... [id=i-01fedbb1e683827db, 00m22s elapsed]
aws_instance.app_server: Still destroying... [id=i-01fedbb1e683827db, 00m32s elapsed]
aws_instance.app_server: Destruction complete after 33s
aws_key_pair.deployer: Creating...
aws_security_group.allow_ssh: Creating...
aws_key_pair.deployer: Creation complete after 0s [id=terraform-key]
aws_security_group.allow_ssh: Creation complete after 2s [id=sg-06a90fa01577a9240]
aws_instance.app_server: Creating...
aws_instance.app_server: Still creating... [00m10s elapsed]
aws_instance.app_server: Creation complete after 15s [id=i-073594417814571ba]

Apply complete! Resources: 3 added, 0 changed, 1 destroyed.
```

 Steps to change an infrastructure :

1. Configuration

Now update the ami of your instance. Change the aws_instance.app_server resource under the provider block in main.tf by replacing the current AMI ID with a new one. Replace "ami 830c94e3" with "ami-08d70e59c07c61a3a".

2. Apply Changes

After changing the configuration, run terraform apply again to see how Terraform will apply this change to the existing resources.

$ terraform apply

 Steps to destroy an infrastructure :

The terraform destroy command terminates resources managed by your Terraform project. This command is the inverse of terraform apply in that it terminates all the resources specified in your Terraform state. It does not destroy resources running elsewhere that are not managed by the current Terraform project.

$ terraform destroy

```
Plan: 0 to add, 0 to change, 3 to destroy.

Do you really want to destroy all resources?
  Terraform will destroy all your managed infrastructure, as shown above.
  There is no undo. Only 'yes' will be accepted to confirm.

  Enter a value: yes

aws_instance.app_server: Destroying... [id=i-073594417814571ba]
aws_instance.app_server: Still destroying... [id=i-073594417814571ba, 00m13s elapsed]
aws_instance.app_server: Destruction complete after 23s
aws_key_pair.deployer: Destroying... [id=terraform-key]
aws_security_group.allow_ssh: Destroying... [id=sg-06a90fa01577a9240]
aws_key_pair.deployer: Destruction complete after 0s
aws_security_group.allow_ssh: Destruction complete after 0s

Destroy complete! Resources: 3 destroyed.
ved@DESKTOP-BHPHDFO:~/learn-terraform-aws-instance$
```

Conclusion :

Successfully built, changed and destroyed an AWS infrastructure using terraform.